Vulnerability report:

UAF In CSS – CVE-2026-2441

© HIGHLION • ALL RIGHTS RESERVED

Table of contents

Background

General Info 3

First Discovery 4

Vulnerability Overview 4

Vulnerability information

Exploit Summary 5

Attack Steps 6, 7, 8

Real World Use 9

Patching the Vulnerability

The Patch 10

Discovery 11

Mitigation 11

Sources and Links 12

© HIGHLION • ALL RIGHTS RESERVED

General Info:

Name: CVE-2026-2441

Severity: 8.8 High

Published Date: 11.2.26

Patch Date: 13.2.26

Affected Platforms: All Chromium based browsers, including: Chrome, Brave, Opera, Edge

Affected Systems: All systems, including Windows, Linux and Mac, as the vulnerability sits in the browser's sandbox and is not affected by the OS

Affected Versions: All versions prior to 145.0.7632.75

Patched versions:145.0.7632.75 and on

© HIGHLION • ALL RIGHTS RESERVED

Initial Discovery:

The Vulnerability was first discovered on11.2.26 by a security researcher named Shaheen Fazim, who created and released the public POC and reported the vulnerability to Google.

It is important to note that at the time of publication, according to the official announcement later made by google, the vulnerability was already being exploited in the wild.

The POC was published to GitHub on the same day, leading to a quick response by the Chromium team.

Vulnerability Overview:

A Use After Free vulnerability is a vulnerability which allows attacker to take control over blocks in the RAM, after they have been “freed” and retuned to the allocator.

CVE-2026-2441 allows for remote code execution (RCE) within the browsers sandboxed process. It was used as a zero-day before being discovered and patched, and the process only required visiting a malicious web page.

© HIGHLION • ALL RIGHTS RESERVED

Exploit Summary:

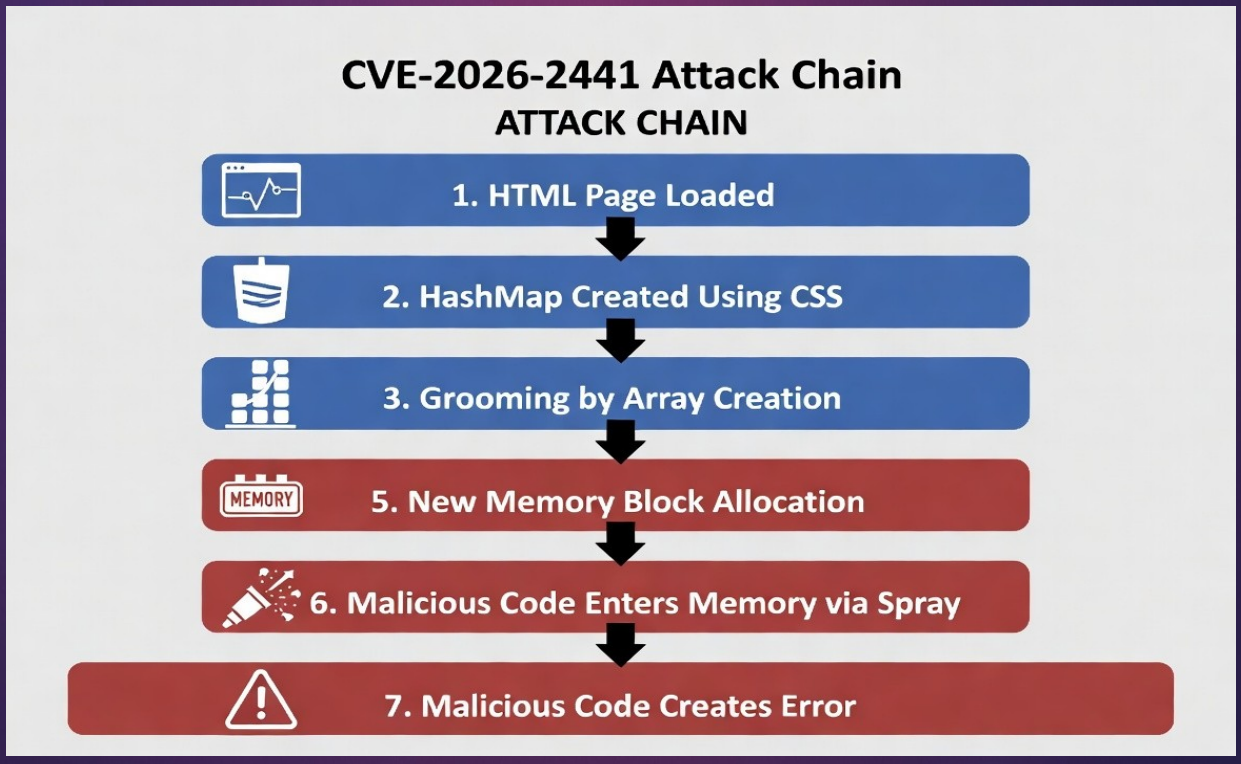
The exploitation is done entirely via JavaScript that runs on the malicious web page.

This attack can come from a few directions, for example an advertisement that loads into the page, or it may be part of a malicious website which is sent as a link in a phishing email.

The essence of the vulnerability sits in the way Blink (Chrome's renderer) reads and handles CSS data when loading a page.

Specifically, in the handling of CSSFontFeatureValuesMap, which is a table of font settings that the browser uses when rendering.

The problem comes from the CSSFontFeatureValuesMap releasing a memory block, while another object known as a pointer, still points to the same memory block after its freed. This access to the memory block after it is no longer being used by the map causes Type Confusion.



© HIGHLION • ALL RIGHTS RESERVED

Attack Stages

- 1) The JavaScript (JS) begins executing immediately when the HTML page is loaded and rendered, and creates the configuration table in CSS.
- 2) Immediately afterward, the JS creates hundreds of Array objects whose purpose is to occupy memory blocks of a specific size. This operation is called heap grooming, and it helps the attacker gain control over a memory block later in the exploit.
- 3) The JS then creates an object called an iterator. Its purpose is to iterate through the CSSFontFeatureValuesMap list mentioned earlier, one entry at a time. The iterator stores, using a pointer, the memory location of this list in case it needs to access it later.
- 4) The JS now causes the memory block allocated for CSSFontFeatureValuesMap to be replaced. This is done by creating and deleting hundreds of entries in the map, causing a larger memory block to be allocated for the same object. The data stored in the map is copied into the new block, while the old block is returned to the allocator and considered free. As a result, the next allocation request may reuse the old memory block.
- 5) In the next stage, the attacker takes control of the freed memory block. The JS rapidly creates objects that imitate entries in the CSS font feature values map, making it likely that one of them will be allocated in the recently freed memory block. This is made possible by the earlier heap grooming: the previously created Array objects occupy memory blocks of that size and shape the allocator's state, increasing the likelihood that the freed block will immediately be reused by an object controlled by the attacker.

© HIGHLION • ALL RIGHTS RESERVED

Attack Stages

- 6) Now that malicious data has been placed into the freed memory block, the attacker attempts to use it. This is done by invoking the iterator created earlier. Throughout the process, the iterator retained the pointer to the original memory location and, due to the software defect, was never updated after the memory block changed. From the iterator's perspective, the data is still located at the old address, where attacker-controlled data now resides.

- 7) As a result, the browser accesses the attacker-controlled data through the stale pointer, resulting in a use-after-free (UAF) condition. The component in Chromium that processes this data (Blink) is implemented in C++. When it interprets attacker-controlled data as if it were the expected internal CSS object, this can lead to type confusion.

- 8) From this point, the attack may proceed in several directions:

  Denial of Service (DoS): A page containing the malicious JS can cause the browser to crash whenever the vulnerable code path is triggered, potentially making the affected web application unavailable.

  Remote Code Execution (RCE): By leveraging the vulnerability, an attacker may achieve arbitrary read/write primitives in the browser process and, under the right conditions, execute arbitrary code.

  Sandbox Escape: An attacker may chain this vulnerability with a separate sandbox escape vulnerability to break out of the browser sandbox and ultimately gain control of the underlying operating system.

© HIGHLION • ALL RIGHTS RESERVED

The POC

- In the following video, we can see the difference in opening a malicious HTML page between a vulnerable version of Chrome, and then a patched version of Edge.

- In the vulnerable browser, the page crashes before it renders, while in the patched browser the page loads cleanly.

© HIGHLION • ALL RIGHTS RESERVED

Real World Use:

- Due to the severity, the type of vulnerability (RCE), and the desire to avoid attacks, no reports of specific campaigns or exploitations were published, however Google announced that it is aware the vulnerability was being exploited in the wild before it became public.

- Such attacks commonly come in the form of malicious links sent in phishing messages. After gaining RCE, an attacker will try to perform other actions in the memory, download payloads, create persistency, and so on.

© HIGHLION • ALL RIGHTS RESERVED

How the patch works:

- The changes were made in the file: css_font_feature_values_map.cc
- The patch avoids using a pointer to access the map, and instead creates a full copy of the data in the CSSFontFeatureValuesMap
- Beyond that, the patch improves handling for memory blocks allocated to the map; It adds verification checks before accessing objects using an iterator or pointer, and prevents freeing of memory blocks which still have active pointers.
- The main change is in the way the map is referenced, as in the vulnerable version the program refers to the pointer by using "*"

```
const FontFeatureAliases* aliases)

: aliases_(aliases),
  iterator_(aliases->begin())

const FontFeatureAliases* aliases_;

return MakeGarbageCollected<FontFeatureValuesMapIterationSource>(
    *this, aliases_);
```

- In the patched version, the object itself (FontFeatureAliases) is called directly:

```
const FontFeatureAliases aliases)

: aliases_(std::move(aliases)),
  iterator_(aliases_.begin())

const FontFeatureAliases aliases_;

return MakeGarbageCollected<FontFeatureValuesMapIterationSource>(
    *this, aliases_ ? *aliases_ : FontFeatureAliases());
```

© HIGHLION • ALL RIGHTS RESERVED

Discovery:

Discovering a potential attack is done mostly with EDR/XDR monitoring:

- Suspicious processes created by a browser session such as powershell.exe or cmd.exe
- Monitoring web traffic to malicious domains, or domains serving suspicious CSS data.
- Monitoring suspicious browser activity, such as a crashing website, or unexpected occurrences in the memory of a browser process.

Mitigation:

- Immediately updating every browser in the network, and as an organizational rule ensuring regular browser updates.
- Closing and restarting sessions after important security updates.
- Blocking execution of unsigned binaries created by browser proccesses.
- Consider blocking JS for browsers on sensitive endpoints.

© HIGHLION • ALL RIGHTS RESERVED

Sources and Links:

The HTML POC File:

- https://github.com/huseyinstif/CVE-2026-2441-PoC/blob/main/poc.html

Chromium source code:

- https://source.chromium.org/chromium/chromium/src/+/main:third_party/blink/renderer/core/css/css_font_feature_values_map.cc

Google's publication of the CVE:

- https://chromereleases.googleblog.com/2026/02/stable-channel-update-for-desktop_13.html

Chromium's publication of the CVE:

- https://issues.chromium.org/issues/483569511

CICA Catalog for monitoring active exploitation:

- https://www.cisa.gov/known-exploited-vulnerabilities-catalog?field_cve=CVE-2026-2441

© HIGHLION • ALL RIGHTS RESERVED